

MUHUZE Global Link

Project Summary, Vision, Work Completed & Pre-Launch Roadmap

Purpose: A structured record of what MUHUZE Global Link is, why it matters, what has been built, and what remains before public launch.

1. What MUHUZE Global Link Is

MUHUZE Global Link is being developed as a digital marketplace and business platform connecting buyers, sellers, service providers and communities through one integrated system. It is intended to grow beyond a simple product-listing website into an ecosystem containing marketplace transactions, seller tools, buyer accounts, wallets, referrals, premium services, notifications, order management and financial accounting.

The long-term direction is to make digital commerce more accessible, organized and scalable, beginning in Rwanda/East Africa and potentially expanding to a wider global market.

2. Benefits and Importance to Society

- **Connects buyers and sellers:** makes product discovery and selling easier.
- **Supports small businesses:** gives individuals and businesses a digital marketplace.
- **Encourages digital commerce:** helps more people participate in structured online buying and selling.
- **Creates economic opportunities:** supports legitimate income opportunities around marketplace activity.
- **Builds local-to-global infrastructure:** creates a platform that can start locally and scale internationally.
- **Improves transaction organization:** links orders, payments, revenue, wallets and withdrawals into a controlled financial flow.

3. Major Areas Already Worked On

Marketplace: product categories, uploads, seller information, locations and buyer

interactions. **Authentication:** backend authentication and JWT-based protected API access.

Profiles and dashboards: buyer/seller account areas and wallet-related views.

Orders: buyer orders and seller order management with Pending, Accepted, Rejected, Shipped, Delivered, Completed and Cancelled states.

Seller revenue flow: product revenue remains pending after payment confirmation and is intended to be released when an order reaches Completed.

Wallet: balances, pending balances, earnings, withdrawals and backend wallet transaction records.

Referrals: referral records, APIs and commission architecture, with automatic referral creation intended during registration.

Premium: frontend premium purchase flow, while financial referral commissions are controlled by the backend.

Revenue accounting: RevenueTransaction with source, accounting amount/currency, revenue rate, revenue amount, referral eligibility, status and seller breakdown.

Withdrawals: backend withdrawal foundation connected to wallet/revenue architecture.

API architecture: shared API client with authenticated Bearer-token handling; referral services use the shared client.

Build/Git: TypeScript/Vite production builds have been successful and Git is being used to preserve milestones.

4. Financial Architecture

The financial design separates responsibilities so frontend features do not directly create or credit money.

Order	Records the marketplace purchase and lifecycle.
Payment status	Records Pending, Paid, Failed or Refunded.
RevenueTransaction	Records accounting/revenue generated by a source.
Seller breakdown	Records seller amount, MUHUZE revenue and seller earning.
ReferralCommission	Records eligible referral commissions.
WalletTransaction	Records wallet movements such as commissions, withdrawals, refunds or adjustments.
Wallet	Represents current financial balances and totals.
Withdrawal	Controls movement of available funds out of the platform.

5. Most Important Pre-Launch Principle

A successful frontend build does not mean the platform is launch-ready. The key milestone is proving the complete financial lifecycle: customer payment → order → revenue → seller earning/referral eligibility → wallet accounting → withdrawal, including cancellation, refund and reversal cases.

The backend should remain the source of truth for financial operations; the frontend should display and request those operations.

6. What Remains Before Public Launch

End-to-end marketplace testing: Create controlled buyer, seller and referrer accounts; place orders and test every order status.

Payment-to-revenue integration: Confirm a Paid order creates the correct revenue record exactly once and cannot duplicate money.

Seller earnings to wallet: Verify seller breakdown calculations and pending-to-available release.

Referral commission testing: Verify eligibility and Level 1/2/3 calculations using controlled transactions.

Refund and reversal testing: Ensure cancellations, refunds and reversals correctly undo financial records without double-crediting.

Withdrawal testing: Test available-balance checks, withdrawal creation, status changes, deductions and failures.

Security audit: Verify JWT protection, ownership checks, unauthorized API access, token handling and sensitive endpoints.

Backend validation: Validate prices, quantities, IDs, currencies and amounts on the server.

Financial reconciliation: For every test, compare Order, RevenueTransaction, ReferralCommission, WalletTransaction and Wallet records.

Production configuration: Replace localhost settings with secure production variables, database, URLs, CORS and secrets.

Logging and error handling: Make financial failures traceable without exposing sensitive information.

Performance cleanup: Review the current large frontend bundle and the Rwanda location dynamic-import warning.

UI/UX final pass: Test desktop/mobile layouts, loading states, empty states, errors, notifications, forms and navigation.

Business/legal readiness: Prepare marketplace rules, seller rules, privacy, terms, refund and withdrawal policies and other applicable requirements.

Final launch acceptance test: Run a controlled complete scenario from registration to purchase, completion, financial release, referral and withdrawal.

7. Recommended Real Test

Use a controlled test with Buyer A, Seller B and Referrer C. Create a known-price product, complete a successful payment, verify the order, complete the seller workflow, inspect RevenueTransaction, seller earning, eligible referral commissions, WalletTransaction and Wallet balances, then perform a controlled withdrawal. Repeat with cancellation/refund/reversal.

8. Current Position

MUHUZE Global Link has moved beyond the early UI/prototype stage. A substantial foundation now exists across frontend, backend, authentication, marketplace, orders, referrals, premium, revenue, wallet and withdrawal architecture.

The project is still in the integration and verification stage. The next major milestone is not simply adding more pages; it is proving that the existing systems work together correctly under real transaction scenarios.

Launch target: a controlled, tested and financially consistent marketplace—not merely a successful

build. 9. Working Rules for the Next Stage

- Change one financial flow at a time.
- Build backend source-of-truth logic first.
- Test with controlled accounts and known amounts.
- Verify database records after every important financial action.
- Never trust a UI number until its backend accounting is verified.
- Keep Git commits clear and recoverable.
- Only move toward public launch after end-to-end tests pass.

Prepared from the MUHUZE Global Link project work and architecture discussed so far.

Yes. Based on everything we have completed in **MUHUZE Global Link**, here is the current summary of the **seller verification requirement**, including how it connects to withdrawals.

1. Why seller verification exists

MUHUZE allows a user to become a seller and upload products **without being verified**.

However:

A seller must be verified before they can withdraw money from their wallet.

This is the rule we implemented in the withdrawal backend.

So the flow is:

Register → Sell → Earn → Verify seller → Withdraw

2. SellerVerification model

We created:

backend/models/SellerVerification.js

Each user gets one verification record.

It contains:

Identity

documentType

documentNumber

documentFront

documentBack

Supported documents:

national_id

passport

driving_license

Personal information

fullName

dateOfBirth

nationality

Address

country

province

district

sector

addressDetails

Phone verification

verified

phone

Review information

reviewedBy

reviewedAt

rejectionReason

Status

The verification can be:

not_submitted

pending

approved

rejected

3. Seller verification controller

We created:

backend/controllers/sellerVerificationController.js

It currently handles four operations.

User: Get verification

GET /api/seller-verification/me

This returns the logged-in user's verification record.

If the user doesn't have one, the backend creates:

status: not_submitted

User: Submit verification

POST /api/seller-verification/submit

The user submits:

documentType

documentNumber

fullName

dateOfBirth

nationality

country

province

district

sector

addressDetails

phone

The backend then changes:

not_submitted

↓

pending

We tested this successfully.

Your test returned:

```
"status": "pending"
```

4. Admin approval

Admin uses:

```
PATCH /api/seller-verification/:userId/approve
```

The route requires:

```
authMiddleware
```

```
+
```

```
adminMiddleware
```

When approved:

```
status = approved
```

and:

```
User.sellerVerified = true
```

We tested this successfully too.

Your response was:

```
Seller verification approved successfully.
```

and then `/me` returned:

```
"status": "approved"
```

5. Admin rejection

Admin can also reject:

```
PATCH /api/seller-verification/:userId/reject
```

with:

```
{
```

```
"rejectionReason": "Reason here"
```

```
}
```

The backend changes:

```
status = rejected
```

and:

```
User.sellerVerified = false
```

The rejection reason is stored in:

```
review.rejectionReason
```

6. Routes we created

The complete seller-verification route structure is:

GET

```
/api/seller-verification/me
```

Authenticated user.

POST

```
/api/seller-verification/submit
```

Authenticated user.

PATCH

```
/api/seller-verification/:userId/approve
```

Admin only.

PATCH

```
/api/seller-verification/:userId/reject
```

Admin only.

7. Authentication protection

We already have:

```
backend/middleware/authMiddleware.js
```

It reads:

```
Authorization: Bearer TOKEN
```

Then verifies the JWT and puts the authenticated user into:

```
req.user
```

We also have:

```
backend/middleware/adminMiddleware.js
```

which checks:

```
req.user.role === "admin"
```

So an ordinary user cannot approve or reject verification.

8. The most important part: withdrawal requirement

We added the verification check to:

```
backend/controllers/withdrawalController.js
```

Before creating a withdrawal, the backend checks:

```
const verification =
```

```
  await SellerVerification.findOne({
```

```
    user: userId,
```

```
  });
```

Then it requires:

```
verification.status === "approved"
```

AND:

```
req.user.sellerVerified === true
```

If either condition isn't satisfied, withdrawal is rejected with:

SELLER_VERIFICATION_REQUIRED

We actually tested this.

Before approval, the withdrawal returned:

Seller verification is required before you can withdraw funds.

So this part is **working correctly**.

9. The complete verification → withdrawal flow

The system now behaves like this:

USER REGISTERS

|

▼

USER CAN SELL PRODUCTS

|

▼

USER EARNS MONEY

|

▼

USER TRIES TO WITHDRAW

|

▼

Is sellerVerified = true?

|

|— NO —> BLOCK WITHDRAWAL

|



Is verification.status = approved?

|

|— NO —> BLOCK WITHDRAWAL

|



ALLOW WITHDRAWAL

|



Create Withdrawal

|



Admin reviews withdrawal

|



Processing

|



Completed

10. What we tested successfully

We tested the whole important section.

Step 1 — Initially unverified

The user had:

status: not_submitted

Withdrawal attempt returned:

SELLER_VERIFICATION_REQUIRED

✔ Correct.

Step 2 — Submit verification

We submitted test information.

Result:

status: pending

✔ Correct.

Step 3 — Admin approval

Admin approved the seller.

Result:

status: approved

and:

sellerVerified: true

✔ Correct.

Step 4 — Withdrawal

After approval we successfully created:

\$5 withdrawal

with:

Fee = \$0.25

Net payout = \$4.75

✓ Correct.

Step 5 — Admin processing

We processed the withdrawal:

Pending

↓

Processing

✓ Correct.

Step 6 — Admin completion

We completed it:

Processing

↓

Completed

✓ Correct.

11. What is NOT finished yet

There are a few things we intentionally haven't fully implemented yet.

A. Real document uploads

Currently:

documentFront

documentBack

are just fields in the database.

We haven't connected actual file upload/storage yet.

So the next real verification feature could be:

National ID front

National ID back

Passport

Driving license

with actual image/PDF upload.

B. Real phone OTP

Currently:

phoneVerification.verified

exists, but the controller explicitly leaves it:

false

until a real OTP system is connected.

So **phone verification is not yet a real verification process.**

C. Admin verification dashboard

The backend approval/rejection functionality works.

But we still need to make sure the admin frontend has a proper interface for:

Pending sellers

Seller details

Documents

Approve

Reject

Rejection reason

Verification status

D. Wallet cleanup

This is separate from seller verification, but we discovered it during withdrawal testing.

Your wallet now has the new:

usd

rwf

structure, but the API is still exposing some old top-level fields:

balance

pendingBalance

totalEarned

totalWithdrawn

currency

We should clean that up.

Also, some wallet transaction creation code still needs the new:

asset

currency

accountingAmount

exchangeRate

fields.

That's why we were examining `walletService.js`.

12. Current status

So I would classify the **seller verification backend** as:

Feature	Status
SellerVerification model	✅ Done
Verification status	✅ Done
User gets verification	✅ Done
Submit verification	✅ Done
Admin authentication	✅ Done
Admin approval	✅ Done
Admin rejection	✅ Done
sellerVerified update	✅ Done
Withdrawal protection	✅ Done
Verification blocks unverified withdrawal	✅ Tested
Approved seller can withdraw	✅ Tested
Real ID document upload	⌚ Not yet
Real phone OTP	⌚ Not yet
Admin verification UI	⌚ Needs completion/check
Wallet migration cleanup	⌚ Current task

In short

The important security rule is already working:

No approved seller verification = no withdrawal.

And we've proven it both ways: **unverified withdrawal was blocked**, then after admin approval, the withdrawal succeeded.